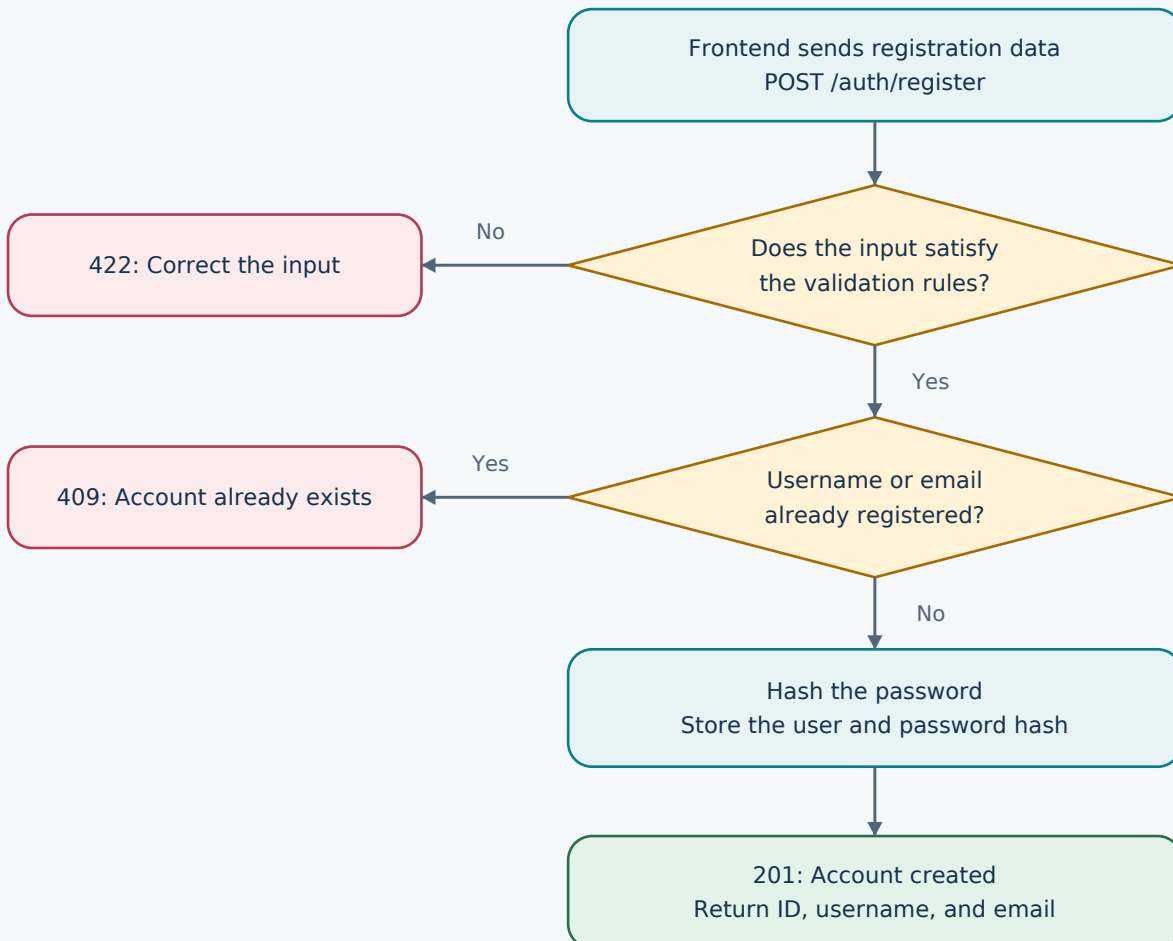


# 1. Register with email and password

Create a new account / POST /auth/register

The frontend is the interface the user interacts with. The backend checks the submitted information. First, Hamza enters a username, email address, and password.



## Example request

```
POST /auth/register
{"username": "Hamza",
 "email": "hamza@example.com",
 "password": "HamzaLearning2026!"}
```

Password: 15-128 characters, uppercase and lowercase letters, a digit, a symbol, and no spaces.

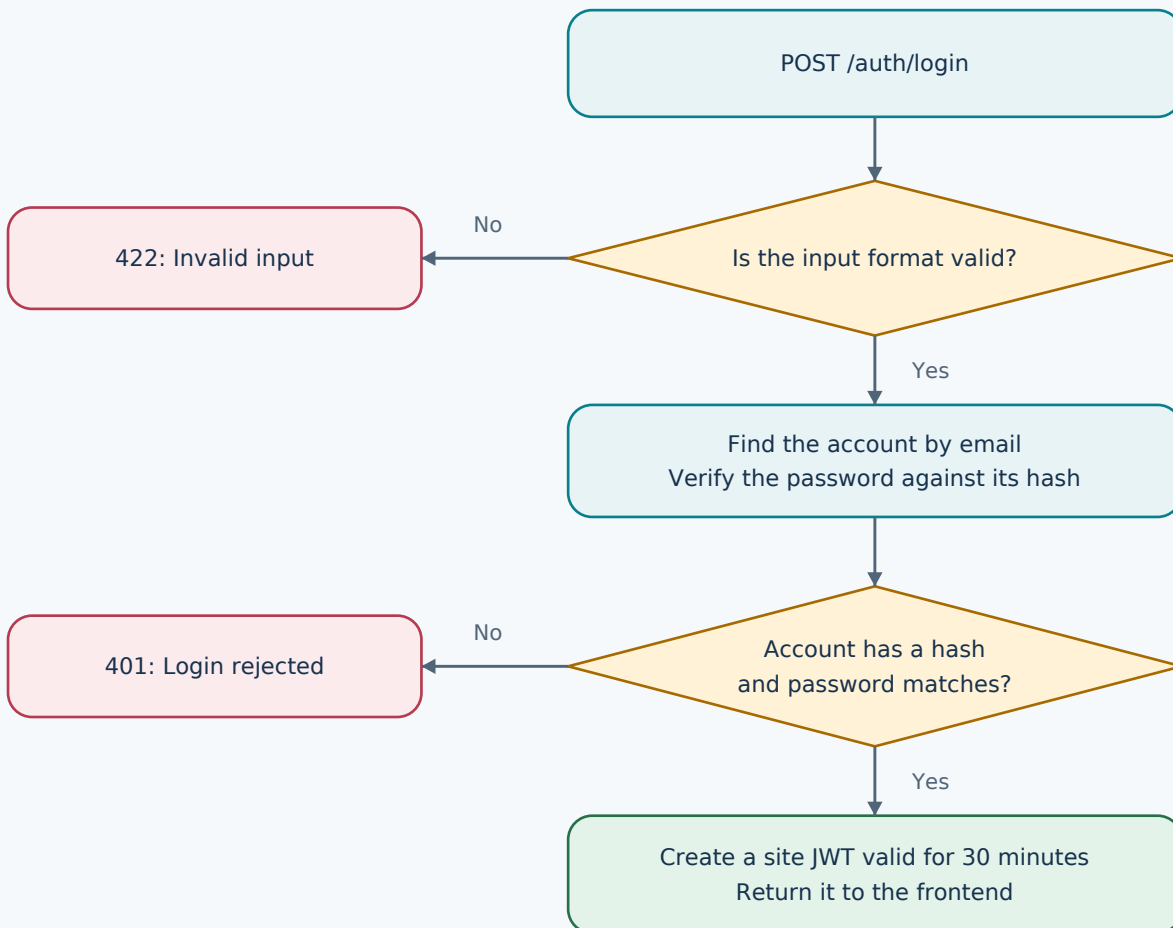
Username: 3-15 characters, begins with a letter, and contains only letters and numbers.

The server stores a hash, not the original password. Hashing is not reversible decryption. Registration here does not return a JWT: the user must log in afterward.

## 2. Log in with email and password

Verify the user / POST /auth/login

Hamza now has an account. He submits his email and password, and the server checks whether they match the stored account.



```
POST /auth/login
{"email": "hamza@example.com",
 "password": "HamzaLearning2026!"}
Response: {"access_token": "SITE_JWT", "token_type": "bearer"}
```

`verify_password()` checks the submitted password against the stored hash using the hashing library. It does not decrypt the hash.

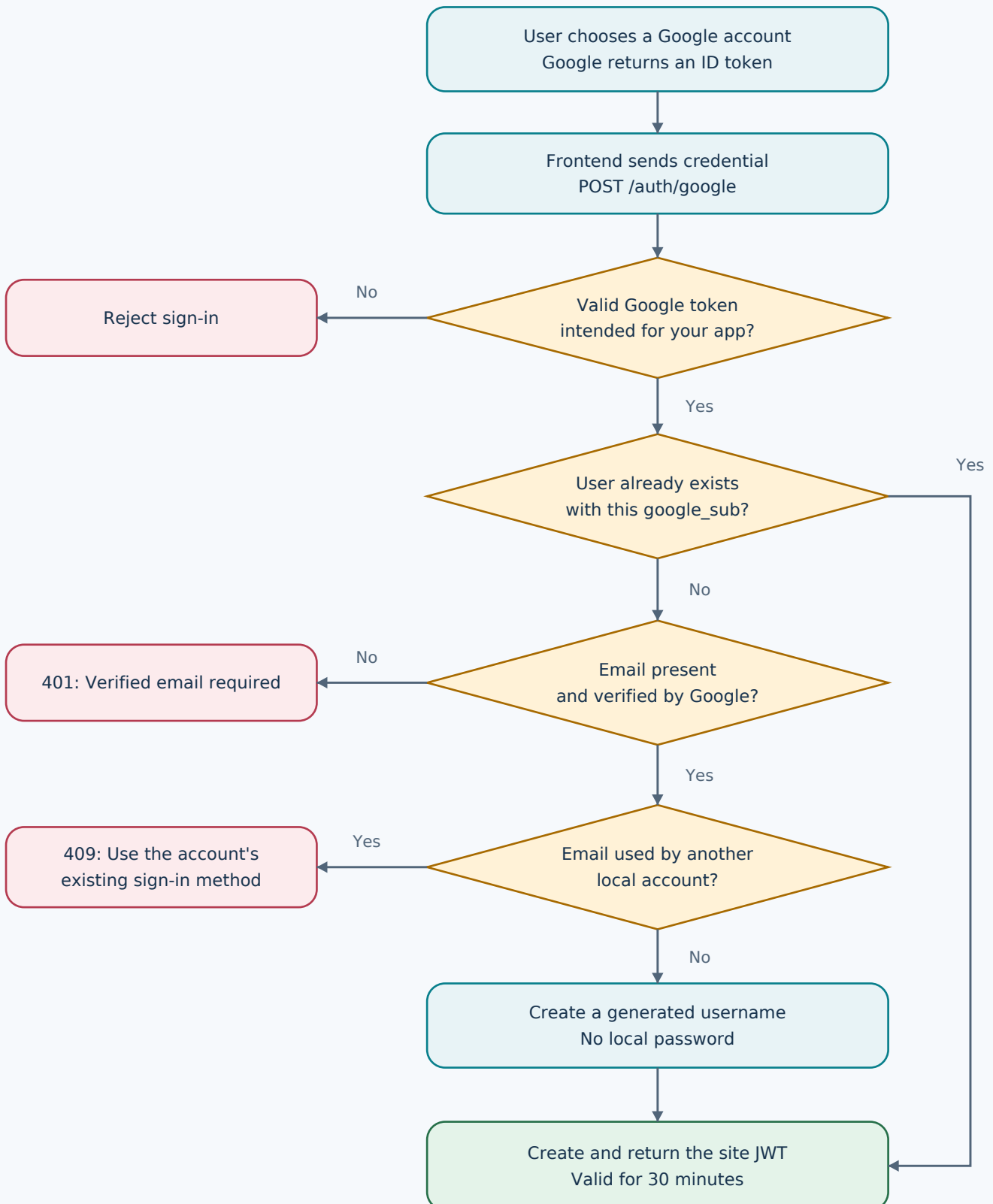
A JWT acts like a temporary entry pass. It contains the local user ID in `sub`, issue time in `iat`, and expiry time in `exp`. The server signs it with its secret; it contains no password.

Hashing and password verification run in a worker thread so this synchronous work does not block the main event loop.

### 3. Sign in with Google

From a verified Google identity to your own site JWT

Google verifies the user. The Google account password never reaches your backend.



# Example: Hamza signs in with Google

Two tokens, with different jobs

Hamza clicks the Google button, chooses an account, and signs in at Google if needed. The frontend receives a token that proves his Google identity.

```
POST /auth/google
Content-Type: application/json

{ "credential": "GOOGLE_ID_TOKEN" }
```

The server checks the token, signature, expiry, and whether it is intended for the configured `GOOGLE_CLIENT_ID`. It reads `sub`: the stable identifier of that Google account.

If no user exists with that `google_sub`, the server requires a verified email that is not already used. It creates a user like this; the local ID is generated separately:

```
username      = "Playera1b2c3d4e"
email         = "hamza@example.com"
google_sub    = "GOOGLE_USER_ID"
password_hash = None
```

It then returns a site token. On the next Google sign-in, it finds Hamza through `google_sub` and issues a new site token without creating another account.

```
Response:
{ "access_token": "SITE_JWT", "token_type": "bearer" }
```

## Do not confuse the tokens

Google ID token: created by Google and sent to `/auth/google` to prove identity. Site JWT: created by your backend and sent with protected requests such as `/auth/me`.

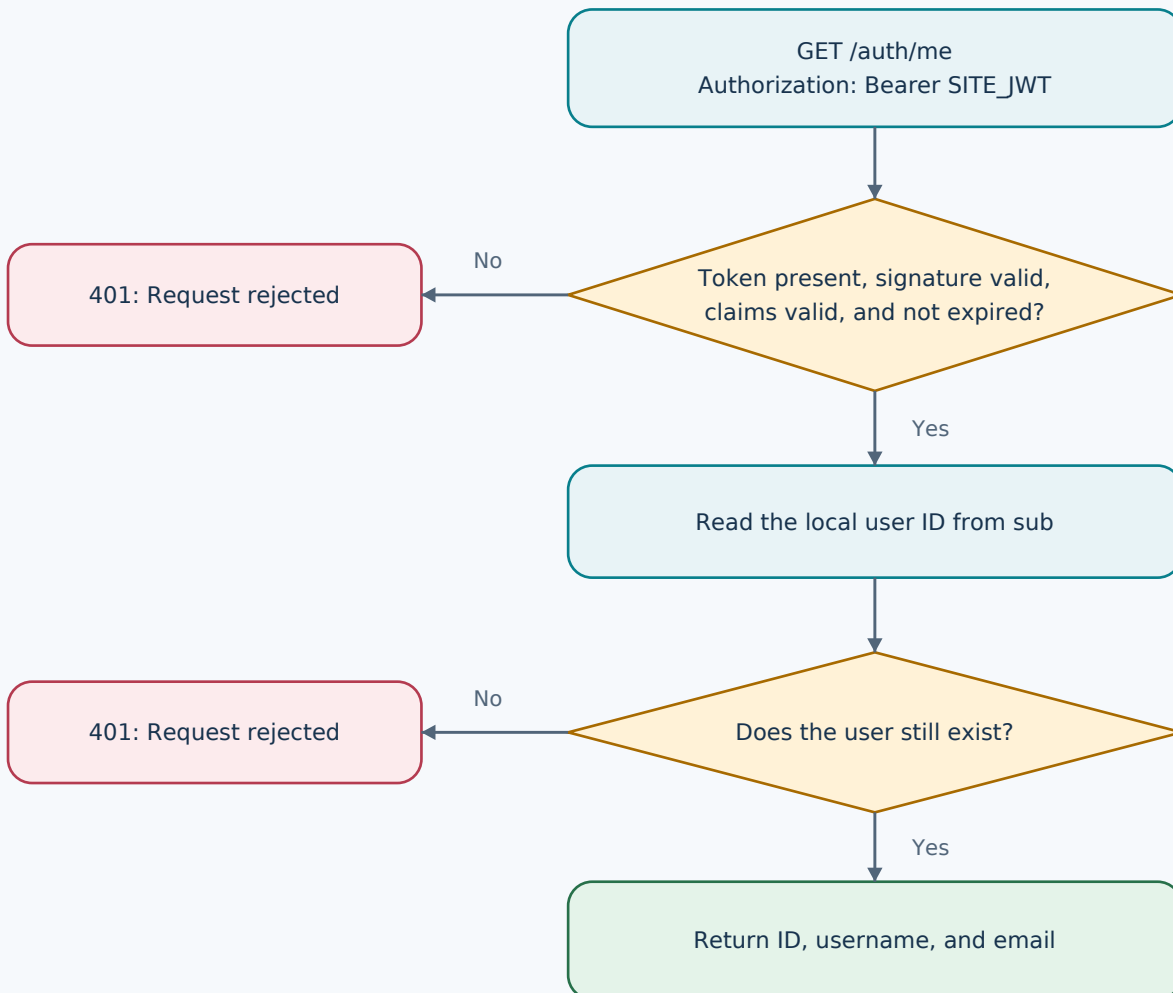
If the email is already registered through the normal method, this code rejects automatic account linking and asks the user to use their existing method.

Your `index.html` is a test page: it displays the Google token for you to copy into `credential` in `/docs`. It does not automatically send it to the backend.

## 4. After login: who are you?

The same protected-request flow after either sign-in method

To retrieve his account details, Hamza sends the site JWT in the Authorization header.



```

GET /auth/me
Authorization: Bearer SITE_JWT
Response: {"id": "USER_UUID", "username": "Hamza",
  "email": "hamza@example.com"}
  
```

The JWT is sent with every request requiring authentication, not just once. /auth/me returns user information, not a new token.

After 30 minutes, the user must sign in again: there is no refresh token in this code. The full frontend storage strategy is also not defined in these files.

Users are stored in TEST\_USERS in memory, not a persistent database. Restarting the server removes the accounts, so an old token will no longer find its user.

# How to read this guide

Pages 1-5: the original flowcharts | Next: concepts and code in detail

The goal is to understand the complete journey: what the user enters, what the frontend sends, what the backend verifies, and what is stored or returned. Read the concepts, then return to the flowcharts and trace each arrow yourself.

## Reading map

| Pages | What you will learn                                                                              |
|-------|--------------------------------------------------------------------------------------------------|
| 1-5   | Registration, password login, Google sign-in, and using the token: four flowcharts and examples. |
| 7-11  | HTTP and FastAPI; hashing and signing; JWTs; cookies and sessions; expiry and logout.            |
| 12-14 | OAuth, OAuth 2.0, OIDC, the authorization code flow, async, and threads.                         |
| 15-22 | Every source file, function, and relevant class: inputs, outputs, errors, and examples.          |
| 23-25 | A complete walkthrough, implementation limits, understanding checks, and answers.                |
| 26-27 | Official references with clickable links.                                                        |

## Three kinds of explanation

"In your code" means behavior actually present in backup.zip. "Illustrative example" uses fictional values to clarify an idea. "Future development" describes something not implemented in the supplied files.

This guide explains the attached version; it does not change the project code. Names such as `SITE_JWT` and `USER_UUID` are placeholders, not valid tokens or IDs you can use unchanged.

To check your understanding, explain each flowchart arrow in your own words. Predict the answers on page 24 before reading the solutions.

# The basics: request and response

Authentication, authorization, HTTP, JSON, and FastAPI

## Who are you? What are you allowed to do?

Authentication verifies identity: are you really Hamza? Authorization checks permissions: now that we know you, may you delete another person's room? Logging in does not grant permission to do everything.

## What is inside a request?

```
POST /auth/login
Content-Type: application/json

{"email": "hamza@example.com",
 "password": "HamzaLearning2026!"}
```

POST is the HTTP method. /auth/login is the path. Headers describe the request; Content-Type says that the body uses JSON. JSON is structured text, converted here into a Python LoginData object. It does not have to be a .json file on disk.

GET /auth/me requests information. The response has a status code and a body. HTTPS encrypts the connection in transit; hashing passwords for storage does not replace HTTPS.

## Who calls the function under @?

A decorator such as `@auth_router.post("/login")` registers the relationship between a route and a function. When the matching request arrives, FastAPI reads the body, validates LoginData, then calls `login(data)`. You do not call it manually.

APIRouter groups routes. `prefix="/auth"` adds their shared prefix. The main application must include `auth_router` with `include_router`. The main application file is absent from this archive, so these files alone cannot confirm that integration.

`Depends(get_current_user)` means: run that dependency first and place its result in `user`. If authentication raises an error, `me` does not proceed to return account information.

# Hashing, encryption, and signing

Three concepts that solve different problems

| Concept         | What it does                                                           | Project example  |
|-----------------|------------------------------------------------------------------------|------------------|
| Hashing         | Produces a one-way result; you do not decrypt it to recover the input. | Password storage |
| Encryption      | Hides content; the appropriate key allows decryption.                  | HTTPS in transit |
| Signature / MAC | Detects changes and verifies that the creator had the required key.    | Site JWT         |

## Why not store the password directly?

If storage leaks, plaintext passwords are immediately exposed. Password hashing increases the cost of guessing. Use a password-specific algorithm: plain SHA-256 is too fast for this purpose. The library manages a random salt and cost settings. [S8]

A salt is not a second password or a secret you must hide. It is random data that lets the same password produce different hashes. It is usually encoded with the hash and its settings. Verification reads that information to check a candidate password. [S8]

```
h1 = hash_password("HamzaLearning2026!")
h2 = hash_password("HamzaLearning2026!")
# Random salts can make h1 and h2 different.
verify_password("HamzaLearning2026!", h1) # True
verify_password("WrongPassword!", h1)     # False
```

## The JWT secret is separate from password hashing

JWT\_SECRET\_KEY signs and verifies your JWTs using HS256. hash\_password does not use that variable: it calls password\_hasher.hash. Changing JWT\_SECRET\_KEY invalidates old token signatures but does not change stored password hashes.

HS256 uses the same secret to create and verify an HMAC. With asymmetric signatures, a private key signs and a public key verifies. Google's verification library handles verification using Google's public keys. [S1]

Sources: Password Storage [S8], Google [S1] | Clickable references on pages 26-27.

# JWT: what is inside a token?

JSON Web Token | This implementation signs tokens; it does not encrypt them

JWT is a format for information called claims. The common signed representation has three dot-separated parts: header, payload, and signature. JWTs can also be encrypted, but `jwt.encode` with HS256 in your code only signs them. [S6]

HEADER.PAYLOAD.SIGNATURE

```
# Illustrative decoded payload:
{
  "sub": "550e8400-e29b-41d4-a716-446655440000",
  "iat": 18000000000,
  "exp": 1800001800
}
```

The header identifies the type and algorithm. The payload contains the claims. The signature protects the preceding parts: if someone changes `sub` to impersonate another user, verification fails. Base64url is readable encoding, not secrecy. [S6]

| Claim            | Meaning in your code                                                              |
|------------------|-----------------------------------------------------------------------------------|
| <code>sub</code> | The local user UUID, not the email address.                                       |
| <code>iat</code> | Issue time. The code uses the current UTC time.                                   |
| <code>exp</code> | Expiry time: 30 minutes after issuance.                                           |
| <code>nbf</code> | The earliest acceptable time, if present. Your creation function does not add it. |

Example: a token issued at 10:00 expires at 10:30. A valid request at 10:10 succeeds; at 10:31 it is rejected. Sending it to `/me` neither extends its lifetime nor renews it.

Bearer means possession of the token allows its use when it is valid and the other checks pass. Someone who steals it may use it until expiry or cancellation through additional logic. Do not put passwords in it or write tokens into logs.

Sources: JWT [S6]; timing behavior from `security.py` | Clickable references on pages 26-27.

# JWT, cookies, and sessions

These are not three alternatives of the same kind

JWT is a token format. A cookie lets a browser store a value and automatically send it with matching requests. A session maintains a user's signed-in state across requests. A cookie can carry a JWT or just a session ID. [S4, S10]

| Comparison   | JWT                                                 | Session ID                                                |
|--------------|-----------------------------------------------------|-----------------------------------------------------------|
| Contents     | Claims and a signature inside the token.            | A random identifier; its meaning is stored on the server. |
| Verification | Verify the signature and required claims.           | Look up the stored session.                               |
| Revocation   | Needs additional logic for cancellation before exp. | Removing the stored session stops its use.                |
| Transport    | Can travel in a header or a cookie.                 | Can travel in a cookie.                                   |

## Session example: the server keeps the record

```
# Server-side session store (illustration)
sessions["random_session_id"] = {"user_id": "USER_UUID"}

# Response header
Set-Cookie: sid=random_session_id; HttpOnly; Secure; SameSite=Lax

# Later request header
Cookie: sid=random_session_id
```

The browser sends sid. The server looks it up to identify the user. The browser's value need not contain the username or account details. [S10]

## What does your project do?

It returns a JWT inside JSON. `get_current_user` reads `Authorization: Bearer`. There is no `Set-Cookie`, cookie reader, or session table in these files. Putting the JWT only in a cookie will not work without changing this code.

Signature verification does not require storing the token, but your code looks up the user on every protected request. It is therefore inaccurate to call the entire system free of stored state.

Sources: Cookies [S4], Session Management [S10] | Clickable references on pages 26-27.

# Token storage and logout

Storage choices, XSS, CSRF, and expiry

| Frontend option | What it means                                                            |
|-----------------|--------------------------------------------------------------------------|
| Memory          | A JavaScript variable; it disappears when the page reloads.              |
| localStorage    | Persists between visits; JavaScript from the same origin can read it.    |
| HttpOnly cookie | JavaScript cannot read its value; the browser can send it automatically. |

The complete frontend storage strategy is not implemented in the archive. The choice must match your frontend and API design. localStorage does not automatically send an Authorization header; frontend code must add it. [S14]

HttpOnly blocks JavaScript from reading the cookie. Secure restricts normal transmission to HTTPS. SameSite restricts certain requests originating from another site context. These settings alone do not make an application immune to attacks. [S9]

## XSS and CSRF, with simple examples

XSS: malicious JavaScript runs inside your site, for example because untrusted content was displayed unsafely. It may read a JavaScript-accessible token, or make requests as the user even with HttpOnly. Safe content handling is still necessary.

CSRF: a malicious site tries to make the browser perform an action on your site using automatically attached login cookies. Appropriate defenses include CSRF tokens, Origin checks, and SameSite. CORS alone is neither authentication nor comprehensive CSRF protection. [S11]

## Logout and renewal in this version

Deleting the JWT from the frontend signs that frontend out. A stolen copy may still be accepted until exp. The supplied code has no logout endpoint, blacklist, or session revocation mechanism.

A refresh token is a separate token that can obtain a new access token. It needs suitable storage, protection, and revocation. None exists here, so the user must sign in again after 30 minutes. Merely changing a password would not invalidate old JWTs in this design.

Sources: Set-Cookie [S9], CSRF [S11], localStorage [S14] | Clickable references on pages 26-27.

# OAuth, OAuth 2.0, and OIDC

Delegated permissions versus identity

OAuth names a family of delegated-authorization protocols. When someone says OAuth, they may mean the family, not specifically version 1.0. OAuth 2.0 is a different version and is not directly compatible with 1.0. [S2, S3]

Example: an app wants to read your photos from another service. You approve a limited permission at that service; the app gets an access token without receiving your password. A scope describes the granted permission, such as read-only access. [S3]

| Comparison    | OAuth 1.0a                                                       | OAuth 2.0                                                    |
|---------------|------------------------------------------------------------------|--------------------------------------------------------------|
| Purpose       | An older delegated-access protocol.                              | An authorization framework with different flows.             |
| Requests      | Common methods sign requests using client and token credentials. | Bearer tokens over HTTPS are common; other extensions exist. |
| Compatibility | You cannot migrate simply by changing a version number.          | A different implementation, not just a new name.             |

## Where does OpenID Connect fit?

OIDC adds an identity layer on top of OAuth 2.0. It defines an ID token so an application can identify the signed-in user. OAuth 2.0 alone does not provide a standard user-identity contract. An API access token must not be confused with an ID token. [S5]

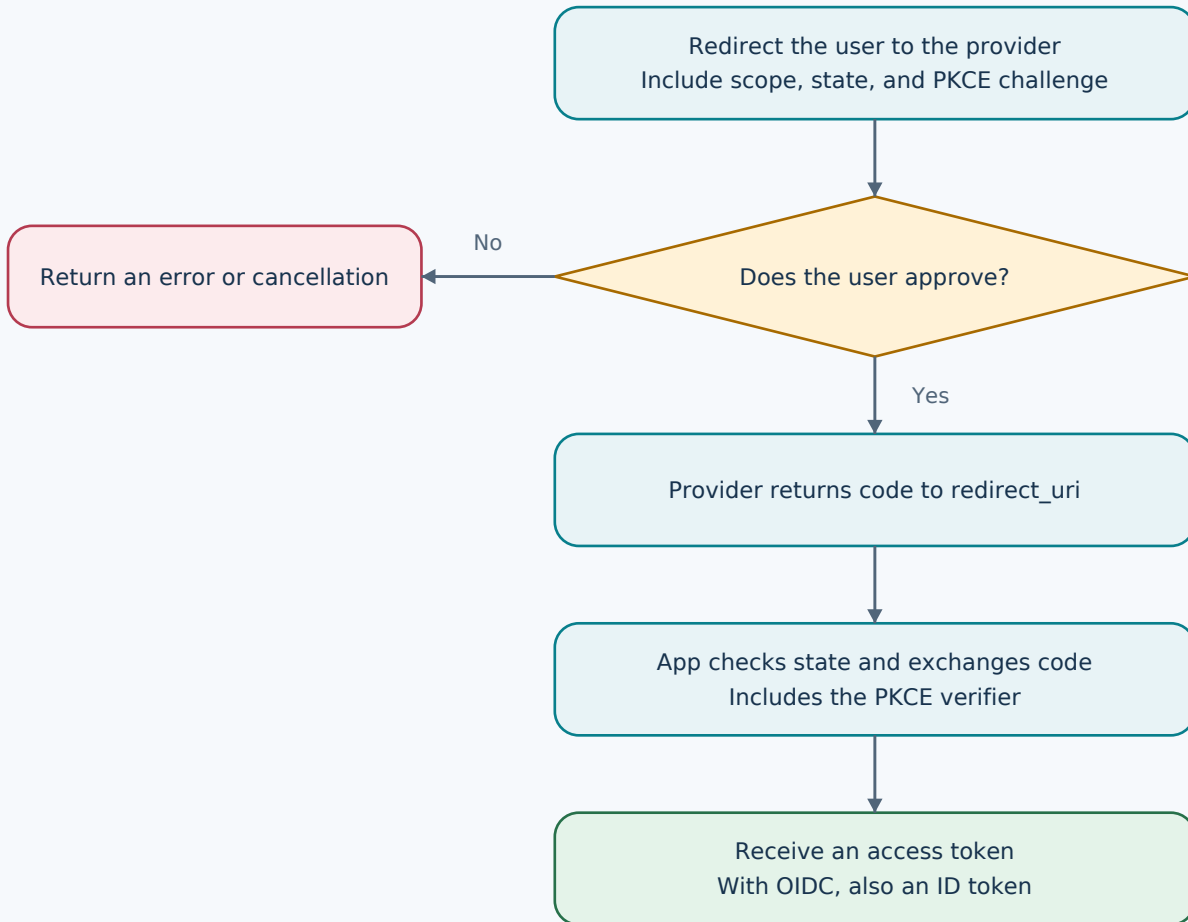
| Token         | What it proves or permits                                      |
|---------------|----------------------------------------------------------------|
| ID token      | The user's identity to the intended application.               |
| Access token  | Access to API resources within granted permissions.            |
| Refresh token | Obtaining a new access token, according to the system's rules. |

In your code, Google Identity Services returns an ID token. Your backend verifies it and issues a local access token. This does not automatically grant your site access to Gmail or Google Drive.

Sources: OAuth 1 [S2], OAuth 2 [S3], OpenID Connect [S5] | Clickable references on pages 26-27.

# Authorization code: another flow

For learning only; this is not the flow implemented in the archive



An authorization code is a short-lived value exchanged for tokens; it is not your site JWT. `redirect_uri` is the registered return address. `state` ties the response to the request you started and helps prevent request mix-ups. [S3]

PKCE creates a random verifier for each attempt and first sends a derived challenge. At exchange time, the app proves it possesses the same verifier, so stealing the code alone is not enough. [S7]

In OIDC, a nonce can bind an ID token to a particular attempt and be checked by the app. Client ID is a public app identifier. A client secret, when used by a confidential backend, must not be placed in frontend code. [S5]

Your `index.html` callback receives `response.credential` directly from Google's library. The archive contains no code exchange, redirect endpoint, or client secret. Do not insert those steps into the actual implementation's flowchart.

Sources: OAuth 2 [S3], PKCE [S7], OIDC [S5] | Clickable references on pages 26-27.

# async, await, and threads

Wait for hashing without blocking the request event loop

`def` defines a regular function. `async def` defines a coroutine function: calling it creates a coroutine that must be awaited or scheduled. `async` alone does not make heavy work lightweight.

`await` pauses this request until a result is ready. During suitable asynchronous waiting, the event loop can serve other requests. It does not mean the user receives the response before hashing finishes. [S13]

```
password_hash = await run_in_threadpool(
    hash_password,
    data.password.get_secret_value(),
)
# This request waits for its hashing result.
```

`run_in_threadpool` runs the regular function in a worker thread. `await` waits without blocking the event loop. Pass the function and its arguments separately; do not call `hash_password(...)` first and pass its already-computed result.

## Example with two requests

| Stage | What happens                                                         |
|-------|----------------------------------------------------------------------|
| 1     | Registration request A submits its hashing work to a thread.         |
| 2     | A waits; the event loop can work on request B for <code>/me</code> . |
| 3     | A's hash is ready; A creates the account and returns its response.   |

Your code uses this pattern in `register`, in `authenticate_user` for hash verification, and in `google_login` for Google token verification, which may involve network access.

A thread pool has limits and consumes CPU and memory. It does not provide infinite capacity or guarantee speed under thousands of requests. The purpose here is to avoid blocking the main event loop with synchronous work.

FastAPI can run a route declared with `def` in a thread pool. But an ordinary `def` function called directly inside `async def` does not automatically move to a thread. You must know where that work runs. [S13]

Sources: FastAPI `async` [S13]; call sites from the project | Clickable references on pages 26-27.

# schemas.py: what input is accepted?

Request and response models, and every validation function

## BaseModel, Field, and SecretStr

BaseModel provides a validated data model. Field adds constraints such as length. EmailStr checks email syntax; it does not prove mailbox ownership. SecretStr masks normal display, not hashing or encryption. `get_secret_value()` returns the original text. [S15]

## RegisterData and LoginData

RegisterData accepts an email up to 256 characters, a username of 3-15 characters, and a password of 15-128 characters with complexity rules. LoginData allows an email up to 256 and a password of 1-128 characters. Login verifies the supplied password rather than reapplying a new-password policy.

## validate\_password\_encoding(value)

Takes SecretStr, attempts to encode its text as UTF-8, and returns the same value on success. Failure raises ValueError. It neither changes nor stores the password. Password = Annotated[...] combines the type with an AfterValidator that runs this check.

## validate\_username(cls, username)

Uses `isalpha` on the first character and `isalnum` on the whole username. `Hamza42` is valid; `42Hamza` and `Hamza_42` are rejected. These Python methods accept Unicode letters, not just A-Z. Valid input is returned unchanged.

## validate\_password\_complexity(cls, value)

Extracts the text from SecretStr and applies `password_policy`: uppercase, lowercase, digits, symbols, and no spaces. Failure raises ValueError; success returns the same SecretStr. `field_validator` attaches it to the field. classmethod supplies `cls` without needing a completed instance.

## UserResponse, TokenResponse, and GoogleLoginData

UserResponse exposes only id, username, and email. TokenResponse exposes `access_token` and `token_type`, which defaults to bearer. GoogleLoginData in `google_auth.py` requires a SecretStr credential at least one character long. This alone does not establish token validity; real verification must follow.

Sources: Secret types [S15]; constraints from `schemas.py` and `google_auth.py` | Clickable references on pages 26-27.

# router.py: the application endpoints

register(), login(), and me()

## Router configuration

APIRouter(prefix="/auth", tags=["AUTH"], route\_class=AuthRoute) groups endpoints under /auth and under AUTH in the documentation. AuthRoute customizes validation-error responses. include\_router(google\_router) makes /google available as /auth/google.

## register(data: RegisterData) -> UserResponse

Input: already-validated registration data. It first checks the username or email; an existing one causes 409. Otherwise, it hashes the password in a thread. create\_user then rechecks duplicates and stores the account.

Why check twice? Another request can register the same name while hashing is awaited. The second check catches that situation in this design. A real database needs unique constraints and suitable operations; a Python check alone does not guarantee protection against every concurrency case.

Success: 201 with id, username, and email only. It returns neither password\_hash nor JWT. If create\_user raises DuplicateUserError, register converts it into HTTP 409.

## login(data: LoginData) -> TokenResponse

Extracts the original password and passes it to authenticate\_user. If the result is None, it returns 401 with the generic message Invalid email or password and WWW-Authenticate: Bearer. Otherwise, it creates a JWT from user["id"] and returns it.

```
user = await authenticate_user(email, password)
if user is None:
    raise HTTPException(status_code=401)
token = create_access_token(user["id"])
# Return TokenResponse(access_token=token)
```

## me(user = Depends(get\_current\_user))

It does not trust a user\_id supplied in the request body. FastAPI runs get\_current\_user and injects the authenticated user. me returns UserResponse fields. A caller cannot choose another identity merely by changing the request body.

# security.py: hashing and login

hash\_password(), verify\_password(), and authenticate\_user()

## Settings read when the module loads

JWT\_SECRET\_KEY comes from the environment. If missing, RuntimeError prevents the module from loading instead of using a default secret. JWT\_ALGORITHM is HS256; ACCESS\_TOKEN\_EXPIRE\_MINUTES is 30. The hasher is created with PasswordHash.recommended().

## hash\_password(password: str) -> str

Takes a plaintext password, calls password\_hasher.hash, and returns a hash string. It does not create an account. register passes its result to create\_user. The installed library determines the hashing algorithm and settings; the code does not set them manually.

## verify\_password(password, password\_hash) -> bool

Passes the candidate password and stored hash to the library. Returns True or False. An unknown hash format causing UnknownHashError returns False. This is not a catch-all for every possible programming error.

## Why DUMMY\_PASSWORD\_HASH exists

It is calculated once at module load from a test password. If an email does not exist, or a Google-only user has no hash, verification still runs against this dummy hash instead of returning immediately. This reduces obvious timing differences that can reveal account existence; it does not guarantee identical timing.

## authenticate\_user(email, password) -> user or None

1) Find the account by email. 2) Read password\_hash if an account exists. 3) Verify in a thread using the real hash or DUMMY\_PASSWORD\_HASH. 4) Return None if the account, hash, or match is missing. Otherwise, return the user dictionary.

```
# A Google-only user has password_hash = None.
# Even a matching dummy password cannot bypass this check:
if user is None or not password_hash or not password_matches:
    return None
```

That last condition matters: the dummy hash is not a login backdoor. Even if its test password matches, an account without a real password hash is still rejected by normal login.

# security.py and current\_user.py

Create a token, validate it, and find the authenticated user

## **create\_access\_token(user\_id: str) -> str**

Gets the current UTC time, builds sub, iat, and exp, then calls `jwt.encode` with `JWT_SECRET_KEY` and `HS256`. It returns a string. It neither stores nor sends the token itself; `login` or `google_login` includes it in the response.

## **decode\_access\_token(token: str) -> user\_id**

Despite its name, this does more than read text. `jwt.decode` verifies the signature with the allowed algorithm, temporal validity, and presence of sub, iat, and exp. Token-specific library errors propagate to the caller. Certain type and value errors are explicitly converted to `InvalidTokenError`.

The code additionally requires iat, exp, and nbf, if present, to be integers, and sub to be a string representing a valid UUID. It returns only `user_id`. It does not look up the account or establish its permissions.

## **HTTPBearer(auto\_error=False)**

Parses Authorization and extracts Bearer credentials. `auto_error=False` allows absent credentials to become `None` so your code can return its custom error. `credentials.credentials` is the JWT string without the word Bearer.

## **unauthorized() -> HTTPException**

Builds a 401 exception with `WWW-Authenticate: Bearer`. It returns the exception object; `raise unauthorized()` actually raises it and stops the request path.

## **get\_current\_user(credentials) -> StoredUser**

Missing credentials: 401. `InvalidTokenError` from decoding: 401. Otherwise, look up the ID with `find_user_by_id`. Missing account: 401. Only success returns a user to me through `Depends`.

```
Authorization: Bearer SITE_JWT
```

```
HTTPBearer -> decode_access_token -> find_user_by_id
           -> StoredUser -> me -> UserResponse
```

Using `from None` when raising an exception suppresses display of the previous exception context in the traceback. It does not mean the function returns `None`.

Sources: `security.py` and `current_user.py` | Clickable references on pages 26-27.

# memory\_store.py: storing users

What each lookup searches for and returns

## TEST\_USERS, StoredUser, and DuplicateUserError

TEST\_USERS is an in-memory list. StoredUser is a TypedDict describing dictionary keys for typing; it is neither runtime Pydantic validation nor a database. NotRequired means a key may be absent; None means a present key may have no value. DuplicateUserError is a custom duplicate-account exception.

| Function                | Search                               | Returns      |
|-------------------------|--------------------------------------|--------------|
| find_existing_user_id   | Username OR email, case-insensitive. | ID or None   |
| find_user_by_email      | Email using casefold.                | User or None |
| find_user_by_id         | Exact local ID.                      | User or None |
| find_user_by_google_sub | First matching google_sub.           | User or None |

casefold supports case-insensitive matching: Hamza and hamza count as duplicates here. Searching does not change the stored value. `user.get("google_sub")` does not raise `KeyError` when that key is absent.

## create\_user(username, email, password\_hash, \*, google\_sub=None)

Rechecks username/email duplicates and, if supplied, google\_sub duplicates. A conflict raises `DuplicateUserError`. Otherwise, it generates an ID with `uuid4`, builds the dictionary, appends it to TEST\_USERS, and returns that dictionary.

The `*` makes `google_sub` keyword-only. `password_hash` may be `None` for a Google account. The `google_sub` key is added only when a value is supplied.

```
user = create_user(
    "Hamza", "hamza@example.com", "PASSWORD_HASH"
)
google_user = create_user(
    "Playera1b2c3d4e", "other@example.com", None,
    google_sub="GOOGLE_USER_ID"
)
```

Lookups scan a list; there are no indexes or persistent storage. Multiple server processes can each have a different list. A shared database requires a separate implementation.

Sources: `memory_store.py` | Clickable references on pages 26-27.

# google\_auth.py: identity verification

verify\_google\_token() and google\_login()

## verify\_google\_token(credential: str) -> dict

Reads GOOGLE\_CLIENT\_ID from the environment, returning 503 if absent. Calls id\_token.verify\_oauth2\_token with Request() and audience=client\_id. The library checks signature, expiry, issuer, and intended audience, then returns the claims. [S1]

Request here is the Google library's HTTP transport helper, not FastAPI's incoming Request. TransportError becomes 503 because verification could not reach its dependency. ValueError or GoogleAuthError becomes 401 for an unacceptable token.

## google\_login(data: GoogleLoginData) -> TokenResponse

First runs verify\_google\_token in a thread. Reads sub from the claims and requires a nonempty string. Looks up the user by google\_sub. If found, proceeds directly to issuing the site token.

For a new Google identity, it requires a nonempty email string and email\_verified exactly True. If another local account uses that email, it returns 409 rather than automatically linking accounts.

Generates a username from Player plus nine characters of uuid4().hex, retrying if needed. Calls create\_user with password\_hash=None and google\_sub. A duplicate detected during creation becomes HTTP 409.

## What is stored, and what is returned?

Stores the local ID, generated username, email, google\_sub, and password\_hash=None. The function does not store or return the Google token. It creates a local JWT using user["id"], not claims["sub"].

```
# Different identifiers, different issuers:
google_claims["sub"]    # Google account identity
user["id"]              # Your local account UUID
site_jwt_payload["sub"] # The local UUID
```

An existing user found by google\_sub does not have their stored email updated in this flow. Google's verified-email flag is not always current proof of ownership for an external email address whose owner changed; production needs an explicit policy. [S1]

Sources: Google verification [S1]; google\_auth.py | Clickable references on pages 26-27.

# validation\_route.py: safe errors

AuthRoute, get\_route\_handler(), and safe\_handler()

Normal validation errors may include the value the user submitted. On authentication routes, that might contain a password or token. AuthRoute wraps processing to return error location, type, and message without the original input.

## get\_route\_handler(self) -> handler function

super().get\_route\_handler() obtains FastAPI's original handler and stores it as original\_handler. The method defines the inner safe\_handler and returns it. It returns a function; it does not execute a request at that return statement.

## safe\_handler(request) -> Response

Awaits original\_handler(request) and returns its response on success. If RequestValidationError occurs, it retains only loc, type, and msg for each error, then returns JSONResponse with status 422.

```
# Simplified validation response:
{
  "detail": [
    {
      "loc": ["body", "username"],
      "type": "string_too_short",
      "msg": "String should have at least 3 characters"
    }
  ]
}
```

loc identifies the error location, here username inside the body. type classifies the error; msg explains it. Exact wording depends on the library version, while the permitted response fields are fixed by your code.

## What does it not catch?

It catches RequestValidationError only. HTTPException responses such as 401 and 409 follow FastAPI's usual handling. Other failures do not all become 422. Removing input from responses also does not automatically protect logs elsewhere in the project.

Callable[[Request], Awaitable[Response]] describes a function accepting a Request and returning something that must be awaited to obtain a Response. Typing helps readers and tools; it does not replace validation logic.

Sources: validation\_route.py | Clickable references on pages 26-27.

# index.html: the Google button

Every callback in the test interface

## googleLoaded() and googleFailed()

googleLoaded runs after the Google script loads: it enables initialization and displays a message. googleFailed reports a loading failure. Both only manage the test UI; neither creates a backend account.

## The submit event handler

Calls event.preventDefault to stop normal form navigation. Reads clientId and trims surrounding whitespace. Checks that it ends with .apps.googleusercontent.com. That is a basic format check, not proof of valid configuration at Google.

## google.accounts.id.initialize(...)

Initializes the library with client\_id and a callback. The same client ID must be configured in the backend's GOOGLE\_CLIENT\_ID. It is a public application identifier; appearing in HTML is normal, unlike exposing JWT\_SECRET\_KEY.

## callback(response)

After successful Google sign-in, copies response.credential into the textarea. This is a Google ID token. The page asks you to copy it manually into credential in /docs. No fetch call sends it to /auth/google in this file.

## renderButton() and the initialized page

renderButton draws the Google sign-in button. The code disables initialize and makes the client ID read-only to prevent changes mid-test. The Google script loads asynchronously; onload and onerror connect success and failure to the handlers above.

## Illustrative addition for a complete frontend

```
const res = await fetch("/auth/google", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify({credential: response.credential})
});
if (!res.ok) throw new Error("Login failed");
const result = await res.json();
// result.access_token is your SITE token.
```

This example assumes the same origin. CORS, secure transport, login-request protection, and token storage still require a production design. Adding fetch alone does not complete that design.

Sources: index.html; the final example is illustrative only | Clickable references on pages 26-27.

# Walkthrough and implementation limits

What actually happens, and what remains future work

## Follow Hamza from registration

At 10:00, register validates, hashes the password, and stores a user. It returns 201 without a token. At 10:01, login finds the matching account and password, then creates a JWT expiring at 10:31. At 10:05, /me verifies the JWT and account existence and returns user details.

At 10:32, /me returns 401 because exp has passed. If the server restarted at 10:10, even an unexpired token would find an empty TEST\_USERS list and get 401, despite a valid signature.

| Status | Meaning                               | Where it appears                                               |
|--------|---------------------------------------|----------------------------------------------------------------|
| 201    | Account created.                      | register                                                       |
| 200    | Request succeeded.                    | Successful login, google, and me                               |
| 401    | Identity could not be verified.       | Wrong password, invalid token, or missing user                 |
| 403    | Known identity, forbidden operation.  | Not implemented in these routes; e.g. an admin-only action     |
| 409    | Conflict with an existing account.    | Duplicate username, email, or identity                         |
| 422    | Input does not satisfy its model.     | Correct the submitted information                              |
| 503    | Service unavailable for this attempt. | Missing Google configuration or verification transport failure |

## What is not implemented in the archive?

Persistent database storage, normal-registration email verification, password recovery, refresh tokens, logout and session revocation, linking Google to an existing account, login rate limits, MFA, and role-based permissions.

The main application, HTTPS, CORS, and deployment settings are also not shown here. These files cannot establish the behavior of the whole project. These are limits of the supplied implementation, not changes made by this guide.

Sources: Walkthrough derived from the supplied code | Clickable references on pages 26-27.

# Check your understanding

Open questions: answer before turning to the next page

- 1) Hamza registers successfully and immediately requests /me without Authorization. Is creating the account enough to pass authentication? Why?
- 2) Two users choose the same password. Must their hashes be identical for verification to work?
- 3) Someone reads a JWT payload and changes sub. Can the modified token pass validation without JWT\_SECRET\_KEY?
- 4) You place the JWT in a cookie but leave get\_current\_user unchanged. Will /me read it?
- 5) A user has a normal account and then tries Google with the same email. What does this implementation return?
- 6) google\_sub contains the Google ID; user["id"] contains a local UUID. Which one becomes sub in the site JWT?
- 7) Does a Google ID token give your site permission to read the user's Gmail messages?
- 8) You delete the token from the frontend. Is another copy on a different device automatically invalidated?
- 9) During await run\_in\_threadpool, has registration finished? Must the event loop stay blocked?
- 10) The server restarts while the token still has 20 minutes left. Why can /me reject it?
- 11) What do SecretStr and EmailStr guarantee? Does the first hash the password, and the second prove email ownership?
- 12) Who calls register? Who supplies the user argument passed into me?

## A practical tracing exercise

Use the Google flowchart and follow four cases: an expired token; a user found by google\_sub; an email already used by a normal account; and a completely new user. Name the function executing at each stage.

# Answers: explain the reason

Connect every answer to its function and flowchart

- 1) No. register does not return a JWT. /me requires credentials; missing Bearer credentials cause 401.
- 2) No. Different salts can produce different hashes for the same password. Verification uses the information encoded in the stored hash.
- 3) No, with the validation shown here. Changing the payload makes the signature invalid. Reading content does not grant the ability to sign new content.
- 4) No. HTTPBearer reads Authorization, not cookies. Cookie-based transport requires changing extraction and designing the associated protections.
- 5) 409 if no account is linked to that google\_sub. There is no automatic email-based linking; the user must use their existing method.
- 6) The local user["id"] becomes the site JWT subject. google\_sub remains the stored connection to the Google identity.
- 7) No. The ID token proves identity. Gmail API access requires separate authorization, scopes, and an appropriate token.
- 8) No. Local deletion changes neither the signature nor exp. Another copy can still work while the account exists and all validation conditions pass.
- 9) Registration has not finished; it waits for the hash. The event loop can handle other requests while the worker thread runs.
- 10) Restarting removes TEST\_USERS. get\_current\_user requires an existing account, not only a valid signature.
- 11) SecretStr masks normal display; it does not hash. EmailStr checks syntax, not mailbox ownership. Each has a limited role.
- 12) FastAPI calls register after matching the route and validating input. Depends(get\_current\_user) supplies me with the user after token verification and lookup.

## A final check without memorization

Explain aloud: where does the password enter? Where is it hashed? Where is a JWT created and how is it sent? What does Google prove? If a step fails, what response follows? Connecting each answer to its function demonstrates practical understanding.

# Official references

Click a reference title to open it | Source guide reviewed: 20 September 2026

Implementation explanations come directly from backup.zip. These references explain the general concepts. The guide distinguishes protocol concepts from the behavior actually present in your code.

## **S1 Google: Verify the Google ID token**

<https://developers.google.com/identity/gsi/web/guides/verify-google-id-token>

## **S2 RFC 5849: OAuth 1.0**

<https://www.rfc-editor.org/rfc/rfc5849>

## **S3 RFC 6749: OAuth 2.0**

<https://www.rfc-editor.org/rfc/rfc6749>

## **S4 MDN: Using HTTP cookies**

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies>

## **S5 OpenID Foundation: How Connect Works**

<https://openid.net/developers/how-connect-works/>

## **S6 RFC 7519: JSON Web Token**

<https://www.rfc-editor.org/rfc/rfc7519>

## **S7 RFC 7636: PKCE**

<https://www.rfc-editor.org/rfc/rfc7636>

## **S8 OWASP: Password Storage**

[https://cheatsheetseries.owasp.org/cheatsheets/Password\\_Storage\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html)

# Official references, continued

Click a reference title to open it | Source guide reviewed: 20 September 2026

## S9 MDN: Set-Cookie

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie>

## S10 OWASP: Session Management

[https://cheatsheetseries.owasp.org/cheatsheets/Session\\_Management\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html)

## S11 OWASP: CSRF Prevention

[https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site\\_Request\\_Forgery\\_Prevention\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html)

## S12 FastAPI: Dependencies

<https://fastapi.tiangolo.com/tutorial/dependencies/>

## S13 FastAPI: Concurrency and async / await

<https://fastapi.tiangolo.com/async/>

## S14 MDN: localStorage

<https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>

## S15 Pydantic: Secret types

<https://docs.pydantic.dev/latest/concepts/types/#secret-types>

## Source files covered

router.py, schemas.py, security.py, current\_user.py, memory\_store.py, google\_auth.py, validation\_route.py, and index.html. Files in `__pycache__` are generated runtime artifacts, not the source being explained.